Deliverable 3

Name: Amalie K. Meltvedt and Maren J. Borgaas

Date: 11.11.2026

GitHub Repository Link

The GitHub repository can be accessed through this link:
https://github.com/amaliemeltvedt/Deliverable-3.git

Importing the classes

In [1]:
```
from character_class import Character, Warrior, Mage
from dice_class import Dice
from items import Weapon
from battle_class import Battle
```

Character class

The `Character` class represents a character in the RPG. It contains the attributes name, max HP, damage and weapon. The name is a string chosen by the player themselves, while the HP max attribute is an integer with a default value of 100. This means the character's hit points cannot surpass the given max. Damage is an integer with a default value of 10, meaning the character deals 10 damage if it does not have a weapon equipped.

For example, a player can create their character by only assigning a value to the `name` attribute. This way, the `Character` object will be automatically assigned the default HP and damage values.

In [2]:
```
# Creating a character
hero = Character("Legolas")

# Displaying the character
print(hero)
hero.introduce()
```

```
Character: Legolas with 100 HP and 10 damage
Hi, my name is Legolas!
```

As seen in the output, our created Character object has the default HP value of 100 and 10 damage. However, the created character can have any value of HP and damage if the player chooses to overwrite the default attribute values.

In [3]:
```
# Creating a character with custom HP and damage
enemy = Character("Sauron", hp_max = 150, damage = 20)
```

```
print(enemy)
```

```
Character: Sauron with 150 HP and 20 damage
```

The `Character` class contains methods for taking and dealing damage. The `take_damage()` method ensures only the character's own HP is affected when attacked, while the `attack()` method calls to the opponent's `take_damage()` method. For example, our Hero can deal damage on their enemy by calling the `attack()` method.

In [4]:
```
# Hero attacks enemy
hero.attack(enemy)

print("After attack:")
print(enemy)

# Enemy attacks hero
enemy.attack(hero)

print("After attack:")
print(hero)
```

```
After attack:
Character: Sauron with 140 HP and 20 damage
After attack:
Character: Legolas with 80 HP and 10 damage
```

The `take_damage()` method could also be used to reduce any amount of HP. To ensure that the character's HP cannot go below zero, we added a lower boundary of 0 HP using the `max()` function.

In [5]:
```
# Reducing a custom amount of HP
hero.take_damage(30)

print("After taking damage:")
print(hero)

# Trying to reduce to a negative number will still result in 0 HP
hero.take_damage(100)

print("After taking too much damage:")
print(hero)
```

```
After taking damage:
Character: Legolas with 50 HP and 10 damage
After taking too much damage:
Character: Legolas with 0 HP and 10 damage
```

A character can also heal itself using the `heal()` method.

In [6]:
```
# Character heals
hero.heal(20)

print("After healing:")
print(hero)
```

```
After healing:
Character: Legolas with 20 HP and 10 damage
```

The `Character` class contains an `is_alive` property to check if the Character is still alive. This will return True if the Character's HP is above 0, and false if it is 0.

In [7]:
```python
print(hero.is_alive)
```

```
True
```

This can be used later in the `Battle` class where the `is_alive` property checks if the characters is alive during the battle. The `Character` class also has a Weapon object as an attribute. If a `Character` is equipped with a weapon, it can change the damage dealt by this character. If the character does not have a weapon, the default damage is used. Examples of how the `Weapon` class is used is given below.

Dice class

The Dice class represents a dice with a specified number of sides. The player can choose the number of sides when creating the dice object.

The class has one attribute, `sides` , which is an interger with a default value of 6.

Dice class has three methods (it also has __ init__() needed for initialising the class):

- `roll()` : Rolls the dice and returns a random number between 1 and the number of sides on the dice.
- `__repr__()` : Defines how the dice object is represented as a string.
- `__call__()` : Makes it possible to use the Dice object as a function by calling `self.roll()` .

We can create a Dice object without defining the number of sides, to controll that the default sides will be 6. We can also define the number of sides to 20 as an example:

In [8]:
```python
# Creating dice objects
dice = Dice()
dice_20 = Dice(20)

print(dice)
print(dice_20)
```

```
Dice(sides=6)
Dice(sides=20)
```

We have ensured that the dice at least must have 2 sides. If the player defines a dice with 1 side, it will give a ValueError.

In [9]:
```python
# Try to create a dice with only one side
try:
    invalid_dice = Dice(sides=1)
except ValueError as error:
    print(error)
```

```
A dice must have at least 2 sides.
```

To test the `roll()` function we can use an example dice with 20 sides, and controll that the function can give different results for each roll:

In [10]:
```
# Rolling dice_20 five times
for i in range(5):
    print(dice_20.roll())
```

```
5
1
17
7
19
```

The method `__repr__()` decides how the object will be represented with string, when we use `repr()`. Again we can use dice_20 as an example:

In [11]:
```
# Printing the representation of dice_20
print(repr(dice_20))
```

```
Dice(sides=20)
```

We have implemented a `__call__()` method in the Dice class. This makes it possible to only write the name of the dice object, for example `dice()`, rather than `dice.roll()`. Both `dice.roll()` and `dice()` will roll the dice.

In [12]:
```
# Rolling the dice using both the roll method and the __call__ method

result_using_roll = dice_20.roll()
print(f"Result using roll: {result_using_roll}")

result_using_call = dice_20()
print(f"Result using __call__: {result_using_call}")
```

```
Result using roll: 6
Result using __call__: 8
```

Weapon class

The Weapon class represents weapons that characters can be equipped with during a battle. These weapons can deal damage to other characters. The class has two attributes, `name` and `damage_dice`. `Name` stores the name of the weapon given by the player, while `damage_dice` stores a Dice object that determines the weapon's damage.

This class uses the method `roll_damage()` which rolls the weapon's damage dice, and returns a damage value.

Composition is used in the Weapon class because a Weapon object contains a Dice object. The Weapon class use the `roll()` method of the Dice class to determine the weapon's damage.

We can create a Dice object and pass it as the damage dice when creating a Weapon object:

In [13]:
```python
# Creating a Dice object with 10 sides
spear_dice = Dice(sides=10)

# Creating a Weapon object containing the Dice object (composition)
spear = Weapon(name="Spear", damage_dice=spear_dice)

print(spear.name)
print(spear.damage_dice)
```

```
Spear
Dice(sides=10)
```

We can then use the `roll_damage()` method on the Weapon object, in this example `spear`, to determine how much damage the weapon deals:

In [14]:
```python
# Rolling the damage dice
damage_by_spear = spear.roll_damage()

print(f"The {spear.name} deals {damage_by_spear} damage.")
```

```
The Spear deals 2 damage.
```

This can be implied on multiple weapons, for example:

In [15]:
```python
# Create different dices
axe_dice = Dice(sides=20)
bow_dice = Dice(sides=12)

# Create weapons using the dices
axe = Weapon("Axe", axe_dice)
bow = Weapon("Bow", bow_dice)

# Roll damage for each weapon
print(f"Axe damage: {axe.roll_damage()}")
print(f"Bow damage: {bow.roll_damage()}")
```

```
Axe damage: 3
Bow damage: 3
```

Character subclasses

The `Character` class contains to subclasses of specialised Characters:

- `Warrior` subclass
- `Mage` subclass

Warrior subclass

The `Warrior` subclass is a defensive specialist subclass, equipped with armor to reduce the incoming damage before taking it. It inherits the attributes from the regular `Character` class, then adds an `armor` attribute. This attribute is an integer with a default value of 5. This means an incoming attack performed by the enemy will be reduced by 5 before it hits the Warrior.

In [16]:
```
#Creating a Warrior
warrior = Warrior("Gimli")

print(warrior)
print(f"{warrior.name} has an armor of {warrior.armor}")
```

```
Character: Gimli with 100 HP and 10 damage
Gimli has an armor of 5
```

If Gimli gets attacked, his armor will reduce the incoming damage by 5.

In [17]:
```
# Gimli takes less damage
print(f"HP before attack: {warrior.hp}")
print(f"Armor reduction: {warrior.armor}")

warrior.take_damage(20)

print(f"Damage taken: {20-5}")
print(f"HP after attack: {warrior.hp}")
```

```
HP before attack: 100
Armor reduction: 5
Damage taken: 15
HP after attack: 85
```

As we can see, Gimli's armor reduced the damage from 20 to 15.

Mage subclass

The Mage subclass is a resource-based attacker which inherits the standard `Character` attributes, but gains two more: `mana` and `mana_max`.

In [18]:
```
# Creating the Mage
mage = Mage("Gandalf")

print(mage)
print(f"{mage.name} has {mage.mana}/{mage.mana_max} mana")
```

```
Character: Gandalf with 100 HP and 10 damage
Gandalf has 30/30 mana
```

Mana is an energy source allowing the Mage to deal a magical spell attack dealing double damage. Such an attack costs 10 mana and does not use any weapon, even if the Mage has one.

In [19]:
```
# Showcasing the magical attack
print(f"{enemy.name} HP before attack: {enemy.hp}")
print(f"{mage.name} mana before attack: {mage.mana}")

mage.attack(enemy)

print(f"{enemy.name} HP after attack: {enemy.hp}")
print(f"{mage.name} mana after attack: {mage.mana}")
```

```
Sauron HP before attack: 140
Gandalf mana before attack: 30
Sauron HP after attack: 120
Gandalf mana after attack: 20
```

If we manually change the Mage's amount of mana to 5, meaning it is below the required amount for a magical attack, it will fall back to the inherited base `attack()` method. The amount of HP the attack will reduce is determined by the weapon (if equipped) and dice throw, or the default damage. This will in turn restore 5 mana.

In [20]:
```
# Changing the mage's mana
mage.mana = 5

print(f"{enemy.name} HP before attack: {enemy.hp}")
print(f"{mage.name} before attack: {mage.mana}")


mage.attack(enemy)

print(f"{enemy.name} HP after attack: {enemy.hp}")
print(f"{mage.name} after attack: {mage.mana}")
```

```
Sauron HP before attack: 120
Gandalf before attack: 5
Sauron HP after attack: 110
Gandalf after attack: 10
```

Battle class

Lastly, we created a Battle class to manage turn-based combat between two `Character` objects. The class has two attributes, `hero` and `enemy`, which both store a `Character` object. In addition to the initial method, it also contains the two methods `play_round()` and `simulate_battle()`. The `play_round()` method describes the actions during a single round of combat, where the `hero` first attacks the `enemy`, and if `enemy` is still alive, then `enemy` will in turn attack `hero`. The `simulate_battle()` method contains a `while` loop that repeats the `play_round()` actions until one character's `is_alive` property returns False.

For example, hero and enemy characters can be made into a `battle` object.

In [21]:
```
# Creating a new final hero with max HP
hero = Character("Frodo")

# Overwriting the enemy variable to make sure Sauron also has max HP
enemy = Character("Sauron", hp_max = 150)

battle = Battle(hero, enemy)

print(battle.hero)
print(battle.enemy)
```

```
Character: Frodo with 100 HP and 10 damage
Character: Sauron with 150 HP and 10 damage
```

In this example, Frodo functions as the hero and Sauron as the enemy. By calling the `play_round()` method, exactly one round of combat will be executed.

In [22]:
```
print(f"Before round: {battle.hero.name} HP: {battle.hero.hp}, {battle.enemy.nam
```

```
battle.play_round()
```

```
Before round: Frodo HP: 100, Sauron HP: 150
After round: Frodo HP: 90, Sauron HP: 140
```

To simulate an entire battle, we can call the `simulate.battle()` method.

In [23]:
```
battle.simulate_battle()
```

```
After round: Frodo HP: 80, Sauron HP: 130
After round: Frodo HP: 70, Sauron HP: 120
After round: Frodo HP: 60, Sauron HP: 110
After round: Frodo HP: 50, Sauron HP: 100
After round: Frodo HP: 40, Sauron HP: 90
After round: Frodo HP: 30, Sauron HP: 80
After round: Frodo HP: 20, Sauron HP: 70
After round: Frodo HP: 10, Sauron HP: 60
After round: Frodo HP: 0, Sauron HP: 50
Sauron has won the battle!
```

As we see, the battle is finished and Sauron unfortunately wins. Assigning a subclass and a weapon to the characters might change the outcome. We can overwrite the `Character` variables, this time giving them a weapon and a subclass.

In [24]:
```
# Creating a dice for the weapons sword and staff
sword_dice = Dice(sides=10)
staff_dice = Dice(sides=12)

# Create Weapon objects
sword = Weapon(name="Sword", damage_dice=sword_dice)
staff = Weapon(name="Staff", damage_dice=staff_dice)

# Create Frodo as a Warrior with a sword
# This creates a new Warrior object, but we give the same hp_max value as before
hero = Warrior(
    name="Frodo",
    hp_max=100,
    armor=10,
    weapon=sword
)

# Create Sauron as a Mage with a magic staff
# This creates a new Mage object, but we give the same hp_max value as before
enemy = Mage(
    name="Sauron",
    hp_max=150,
    mana=50,
    weapon=staff
)

# Display the character information
print("Battle participants:")
print(f"{hero.name}: Warrior with {hero.hp} HP, "
      f"{hero.armor} armor and a {hero.weapon.name}")

print(f"{enemy.name}: Mage with {enemy.hp} HP, "
      f"{enemy.mana} mana and a {enemy.weapon.name}")
```

```
Battle participants:
Frodo: Warrior with 100 HP, 10 armor and a Sword
Sauron: Mage with 150 HP, 50 mana and a Staff
```

Now that we have created characters with subclasses and weapons, we can again use `play_round()` to simulate one round of combat.

In [25]:
```python
battle = Battle(hero, enemy)

# Display status before one round
print("\nBefore round:")
print(f"Frodo HP: {hero.hp}, Armor: {hero.armor}")
print(f"Sauron HP: {enemy.hp}, Mana: {enemy.mana}")

# Play one round
battle.play_round()
```

```
Before round:
Frodo HP: 100, Armor: 10
Sauron HP: 150, Mana: 50
After round: Frodo HP: 90, Sauron HP: 141
Sauron Mana: 40
```

After one round we see that Frodo lost exactly 10 HP, while Sauron's mana has decreased with 10. This means he used his mana energy to double the base damage (5) dealt on Frodo. The damage Frodo dealt on Sauron will vary depending on the dice throw because he has a weapon equipped. In a full battle, Sauron's mana will run out and his base attack will also vary depending on a dice throw.

We can simulate the entire battle to find a winner using the `simulate_battle()` method:

In [26]:
```python
# Simulate the entire battle
battle.simulate_battle()
```

```
After round: Frodo HP: 80, Sauron HP: 137
Sauron Mana: 30
After round: Frodo HP: 70, Sauron HP: 133
Sauron Mana: 20
After round: Frodo HP: 60, Sauron HP: 132
Sauron Mana: 10
After round: Frodo HP: 50, Sauron HP: 130
Sauron Mana: 0
After round: Frodo HP: 50, Sauron HP: 123
Sauron Mana: 5
After round: Frodo HP: 50, Sauron HP: 119
Sauron Mana: 10
After round: Frodo HP: 40, Sauron HP: 110
Sauron Mana: 0
After round: Frodo HP: 40, Sauron HP: 108
Sauron Mana: 5
After round: Frodo HP: 40, Sauron HP: 98
Sauron Mana: 10
After round: Frodo HP: 30, Sauron HP: 89
Sauron Mana: 0
After round: Frodo HP: 30, Sauron HP: 80
Sauron Mana: 5
After round: Frodo HP: 30, Sauron HP: 71
Sauron Mana: 10
After round: Frodo HP: 20, Sauron HP: 70
Sauron Mana: 0
After round: Frodo HP: 20, Sauron HP: 63
Sauron Mana: 5
After round: Frodo HP: 20, Sauron HP: 54
Sauron Mana: 10
After round: Frodo HP: 10, Sauron HP: 48
Sauron Mana: 0
After round: Frodo HP: 8, Sauron HP: 38
Sauron Mana: 5
After round: Frodo HP: 8, Sauron HP: 28
Sauron Mana: 10
After round: Frodo HP: 0, Sauron HP: 26
Sauron Mana: 0
Sauron has won the battle!
```

As the damage dealt by the weapon equipped is decided by a random dice throw, the results will vary.